

Task 2 — House Price Prediction

Notebook Link:

<https://colab.research.google.com/drive/1XaDCAE-z9YMhHVbGKM90kyuyz5hx39Ib?usp=sharing>

Summary:

Objective

The objective of this project is to build a machine learning model that can predict house prices based on different housing features and property details.

Dataset Used

The House Price dataset containing features such as:

- Area
- Number of Bedrooms
- Bathrooms
- Stories
- Parking
- Furnishing Status
- Price

Algorithms Used

- Linear Regression
- Random Forest Regressor
- Gradient Boosting Regressor

Steps Performed

- Imported and analyzed the dataset
- Checked and handled missing values
- Encoded categorical features
- Performed feature scaling
- Conducted Exploratory Data Analysis (EDA)
- Visualized correlations using heatmaps
- Split dataset into training and testing sets
- Trained multiple regression models
- Compared model performances
- Performed house price prediction
- Saved trained model using Joblib

Visualizations Included

- Correlation Heatmap

- House Price Distribution Plot
- Residual Analysis Plot

Evaluation Metrics

- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- R2 Score

Result

Among the trained models, Random Forest Regressor achieved better prediction performance with lower error values and better accuracy in predicting house prices.

Tools and Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Google Colab

Screenshots:

+ <> + T

Connect ^

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront
0	2014-05-02 00:00:00	313000.0	3.0	1.50	1340	7912	1.5	
1	2014-05-02 00:00:00	2384000.0	5.0	2.50	3650	9050	2.0	
2	2014-05-02 00:00:00	342000.0	3.0	2.00	1930	11947	1.0	
3	2014-05-02 00:00:00	420000.0	3.0	2.25	2000	8030	1.0	
4	2014-05-02 00:00:00	550000.0	4.0	2.50	1940	10500	1.0	

[]

df.info()

v

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   date                   4600 non-null  object
1   price                  4600 non-null  float64
2   bedrooms               4600 non-null  float64
3   bathrooms              4600 non-null  float64
4   sqft_living            4600 non-null  int64
5   sqft_lot               4600 non-null  int64
6   floors                 4600 non-null  float64
7   waterfront             4600 non-null  int64
8   view                   4600 non-null  int64
9   condition              4600 non-null  int64
10  sqft_above             4600 non-null  int64
11  sqft_basement          4600 non-null  int64
12  yr_built               4600 non-null  int64
13  yr_renovated           4600 non-null  int64
14  street                 4600 non-null  object
15  city                   4600 non-null  object
16  statezip               4600 non-null  object
17  country                4600 non-null  object
dtypes: float64(4), int64(9), object(5)
memory usage: 647.0+ KB
```

[]

df.describe()

v

	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors
count	4.600000e+03	4600.000000	4600.000000	4600.000000	4.600000e+03	4600.000000
mean	5.519630e+05	3.400870	2.160815	2139.346957	1.485252e+04	1.512065
std	5.638347e+05	0.908848	0.783781	963.206916	3.588444e+04	0.538288
min	0.000000e+00	0.000000	0.000000	370.000000	6.380000e+02	1.000000
25%	3.228750e+05	3.000000	1.750000	1460.000000	5.000750e+03	1.000000
50%	4.609435e+05	3.000000	2.250000	1980.000000	7.683000e+03	1.500000
75%	6.549625e+05	4.000000	2.500000	2620.000000	1.100125e+04	2.000000
max	2.659000e+07	9.000000	8.000000	13540.000000	1.074218e+06	3.500000

[]



df.isnull().sum()



...

0

date

0

price

0

bedrooms

0

bathrooms

0

sqft_living

0

sqft_lot

0

floors

0

waterfront

0

view

0

condition

0

sqft_above

0

sqft_basement

0

yr_built

0

yr_renovated

0

street

0

city

0

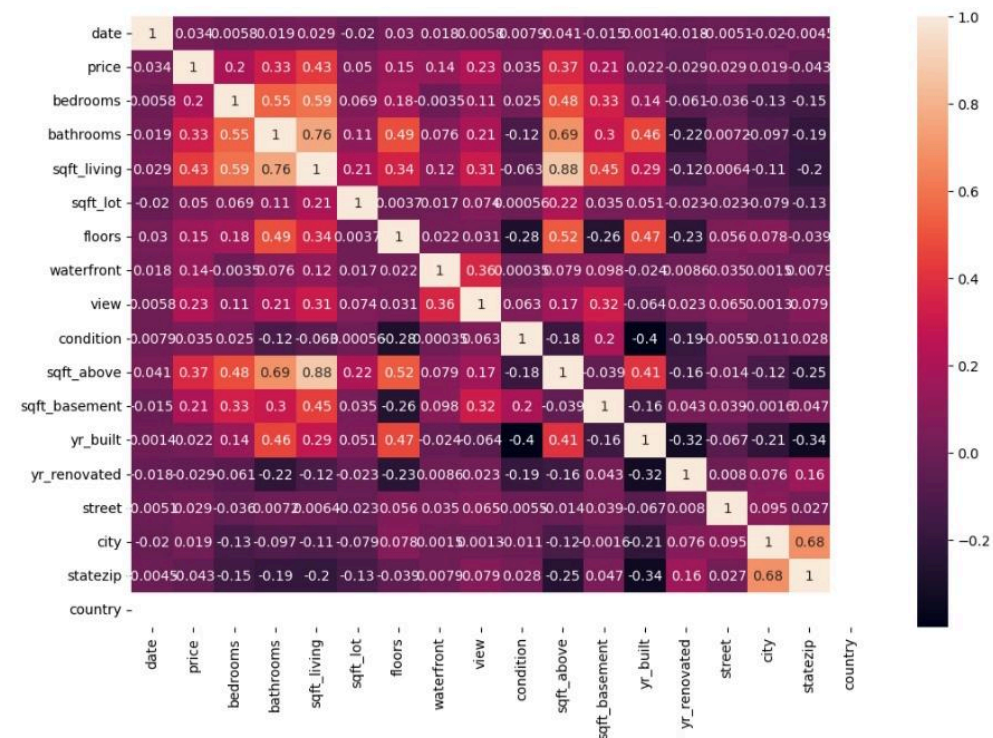
statezip

0

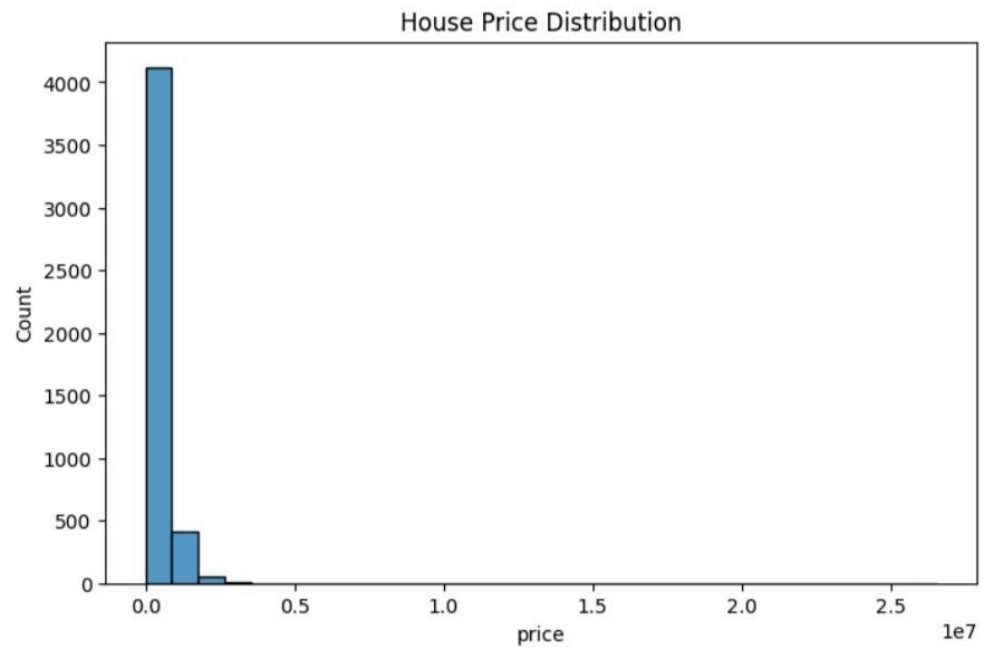
country

0

dtype: int64



▼



▼

